

```
In [3]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

df = pd.read_csv("data.csv")
print("First 5 rows: \n")
print(df.head())
print("\nShape: ", df.shape)
```

First 5 rows:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean \
0	842302	M	17.99	10.38	122.80	1001.0
1	842517	M	20.57	17.77	132.90	1326.0
2	84300903	M	19.69	21.25	130.00	1203.0
3	84348301	M	11.42	20.38	77.58	386.1
4	84358402	M	20.29	14.34	135.10	1297.0

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean \
0	0.11840	0.27760	0.3001	0.14710
1	0.08474	0.07864	0.0869	0.07017
2	0.10960	0.15990	0.1974	0.12790
3	0.14250	0.28390	0.2414	0.10520
4	0.10030	0.13280	0.1980	0.10430

	texture_worst	perimeter_worst	area_worst
0	17.33	184.60	2019.0
1	23.41	158.80	1956.0
2	25.53	152.50	1709.0
3	26.50	98.87	567.7
4	16.67	152.20	1575.0

	compactness_worst	concavity_worst	concave points_worst
0	0.6656	0.7119	0.2654

0.4601			
1	0.1866	0.2416	0.1860
0.2750			
2	0.4245	0.4504	0.2430
0.3613			
3	0.8663	0.6869	0.2575
0.6638			
4	0.2050	0.4000	0.1625
0.2364			

	fractal_dimension_worst	Unnamed: 32
0	0.11890	NaN
1	0.08902	NaN
2	0.08758	NaN
3	0.17300	NaN
4	0.07678	NaN

[5 rows x 33 columns]

Shape: (569, 33)

```
In [7]: #Data prprocessing
from sklearn.preprocessing import LabelEncoder, StandardScaler
drop_cols = ["id","Unnamed: 32"]
valid_drop_cols = [];
for col in drop_cols:
    if col in df.columns:
        valid_drop_cols.append(col)
df = df.drop(columns =valid_drop_cols)

#Encoding target class that is M = 1 and B = 0
le = LabelEncoder()
df['diagnosis'] = le.fit_transform(df['diagnosis'])
print("\nEncoded Classes:")
print(df['diagnosis'].value_counts())
```

```
Encoded Classes:
diagnosis
0    357
1    212
Name: count, dtype: int64
```

```

In [18]: from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X = df.drop(columns=['diagnosis'])
y = df['diagnosis']

print("Feature Shape:", X.shape)
print("Target Shape:", y.shape)

# Split
X_train, X_test, y_train, y_test =
train_test_split(X,y,test_size=0.2,random_state=42)
# Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Default prediction (threshold = 0.5)
y_pred = model.predict(X_test)
# Probability of class 1
y_prob = model.predict_proba(X_test)[:,:1]
# Custom threshold
threshold = 0.3

# Threshold tuning
y_pred_new = (y_prob >= threshold).astype(int)
print("\nThreshold:", threshold)

print("\nDefault Predictions:")
print(y_pred)

print("\nThreshold Predictions:")
print(y_pred_new)

print("\nProbabilities:")
print(y_prob)

```

Feature Shape: (569, 30)

Target Shape: (569,)

Threshold: 0.3

Default Predictions:

```

[0 1 1 0 0 1 1 1 0 0 0 1 0 1 0 1 0 0 0 1 0 0 1 0 0 0 0 0 0 1 0 0 0
0 0 0 1
 0 1 0 0 1 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 1 1 0 0 0 1 1 0 0
1 1 0 1
 0 0 0 0 0 0 1 0 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 0 1 1 0 1 1 0 0 0
1 0 0 1
 0 1 1]

```

Threshold Predictions:

```
[0 1 1 0 0 1 1 1 1 0 0 1 0 1 0 1 0 0 0 1 0 0 1 0 0 0 0 0 0 1 0 0 0
0 0 0 1
 0 1 0 0 1 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 1 1 0 0 1 1 0 0 0 1 1 0 0
1 1 0 1
 0 0 0 1 0 0 1 0 1 1 1 1 1 1 0 0 0 1 0 0 0 0 1 1 0 1 1 0 1 1 0 0 0
1 0 0 1
 0 1 1]
```

Probabilities:

```
[1.13590250e-01 9.99990991e-01 9.96920905e-01 5.10134225e-04
6.06389713e-05 1.00000000e+00 9.99999998e-01 9.65096937e-01
3.79342140e-01 7.59402734e-04 4.98717826e-02 9.87592879e-01
6.07381873e-03 8.56069117e-01 1.62175599e-03 9.99259818e-01
1.85245908e-03 1.11662359e-05 8.18375684e-07 9.99998883e-01
8.41633901e-02 9.74778299e-03 9.99999996e-01 7.89186669e-05
1.13874333e-03 5.84587240e-04 1.07444521e-03 5.97518345e-03
2.85311713e-03 9.99991817e-01 5.17409813e-04 1.48760128e-04
2.16094439e-03 5.49413839e-03 1.40706288e-04 2.42334266e-03
9.42415985e-01 2.89870352e-03 9.99911214e-01 4.81363757e-02
7.12433090e-05 9.99593392e-01 2.46640626e-03 9.63884505e-04
1.99892051e-02 7.15547342e-02 3.98571939e-04 4.90625569e-04
3.04698010e-02 6.14016147e-04 9.99879654e-01 9.99999947e-01
3.08211520e-01 1.32293769e-02 2.73627242e-05 9.54524359e-03
1.12878558e-04 1.00000000e+00 8.35573090e-01 1.39239947e-04
5.01360363e-03 9.99999551e-01 9.99999999e-01 3.11060472e-02
8.29093869e-04 1.29799229e-01 9.99986074e-01 9.99999998e-01
8.08574976e-04 2.03813936e-02 9.98672791e-01 9.98391518e-01
2.57408198e-03 9.95464446e-01 8.88214751e-05 1.07439170e-02
1.99896261e-02 4.84826685e-01 2.73957683e-05 1.38486719e-02
9.95694507e-01 6.41710880e-05 7.76680319e-01 9.99999990e-01
9.96105761e-01 9.98885757e-01 9.99862833e-01 9.99859961e-01
2.19094654e-04 3.40785200e-03 5.60602140e-03 3.22718096e-01
3.42520227e-02 2.87072455e-04 3.90422253e-04 1.31057534e-04
9.99999327e-01 9.99999017e-01 8.17839564e-05 9.99669765e-01
9.96854238e-01 1.58369703e-06 9.99983538e-01 9.99398270e-01
9.90653170e-03 3.49633955e-02 5.75366578e-03 1.00000000e+00
1.13099081e-01 3.21954313e-02 9.99323175e-01 1.30828400e-03
5.42699821e-01 1.00000000e+00]
```

```
In [20]: from sklearn.metrics import confusion_matrix
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import roc_auc_score
# Probability of class 1
y_prob = model.predict_proba(X_test)[:,-1]
# Predictions
y_pred = model.predict(X_test)
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nPrecision:",
      precision_score(y_test, y_pred))
print("\nRecall:",
      recall_score(y_test, y_pred))
print("\nROC AUC:",
      roc_auc_score(y_test, y_prob))
```

Confusion Matrix:

```
[[70  1]
 [ 2 41]]
```

Precision: 0.9761904761904762

Recall: 0.9534883720930233

ROC AUC: 0.99737962659679

SIGMOID FUNCTION - > The sigmoid function is an S-shaped mathematical function used in Logistic Regression to convert the linear output $z=wx+b$ into a probability between 0 and 1.